

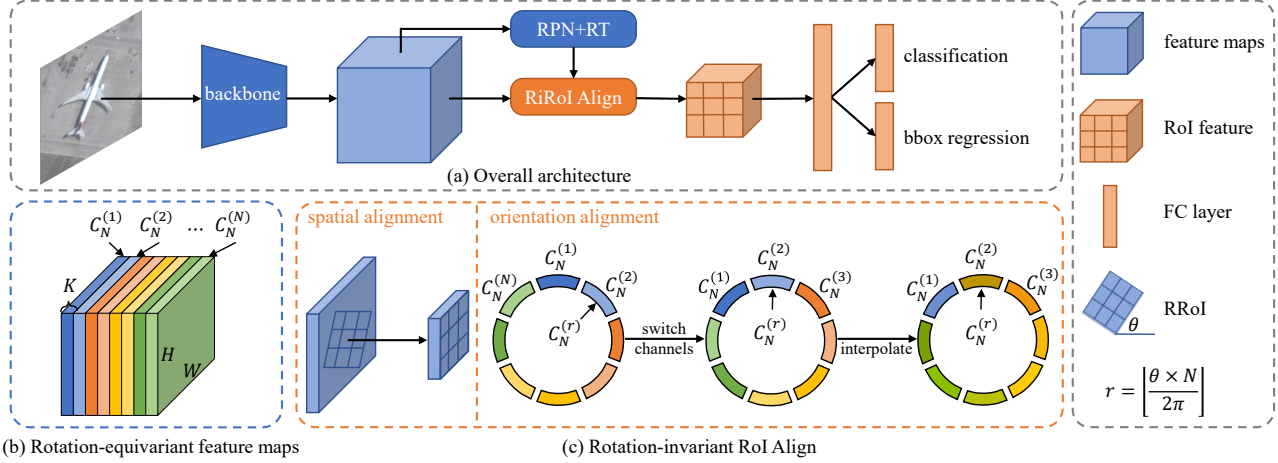


Figure 3. **Overview of our proposed method.** (a) Overall architecture of the proposed Rotation-equivariant Detector. We first adopt the rotation-equivariant backbone to extract rotation-equivariant features, followed by an RPN and RoI Transformer (RT) [7] to generate RRoIs. Then we use a novel Rotation-invariant RoI Align (RiRoI Align) to produce rotation-invariant features for RoI-wise classification and bounding box (bbox) regression. (b) Rotation-equivariant feature maps. Under the cyclic group C_N , the rotation-equivariant feature maps with the size (K, N, H, W) have N orientation channels, and each orientation channel is corresponding to an element in C_N . (c) RiRoI Align. The proposed RiRoI Align consists of two parts: spatial alignment and orientation alignment. For an RRoI (x, y, w, h, θ) , spatial alignment warps the RRoI from the spatial dimension, while orientation alignment circularly switches orientation channels and interpolates features to produce completely rotation-invariant features.

group G . For a layer $L_i \in \Phi$, the rotation transformation T_r can be preserved by the layer:

$$L_i[T_r(g)] = T_r[L_i(g)] \quad g \in G. \quad (4)$$

If we apply T_r to the input I and feed it to the network Φ , the transformation T_r ² will be preserved by the whole network:

$$\left[\prod_{i=1}^M L_i \right] (T_r I) = T_r \left[\prod_{i=1}^M L_i \right] (I). \quad (5)$$

Rotation-invariant Features. For any rotation transformations T_r applied to the input, if its output remains unchanged, we say the output feature is rotation-invariant. Rotation-invariant features can be divided into three levels: image-level, instance-level, and pixel-level. Here we mainly focus on the *instance-level rotation-invariant feature*, which is more suitable for the object detection task. Let $I_R \in I$ and $f_R \in f$ denotes an RoI of the image I and feature maps f ($f = \Phi(I)$), respectively. Assume I_R is a HRoI (x, y, w, h) invariant to the orientation, where (x, y) , w and h denote the center point, width and height of the HRoI, respectively. While $T_r I_R$ is an RRoI (x, y, w, h, θ) related to the orientation θ . Similar to Eq. 5, for RoI I_R , the rotation equivariance can be expressed as:

$$\Phi(T_r I_R) = T_r \Phi(I_R). \quad (6)$$

²The transformation T_r may have different formulations in different spaces, e.g., the input (image) space and the feature space. Here we do not distinguish it for simplicity. For a deeper discussion of rotation-equivariant networks, we refer the readers to [5] and [33].

If we regard HRoI I_R as the rotation-invariant representation of RRoI $T_r I_R$ in the image I , $\Phi(I_R)$ can be regarded as the rotation-invariant representation of $\Phi(T_r I_R)$ in the corresponding feature space. To get $\Phi(I_R)$, we need to know the rotation transformation T_r . Fortunately, T_r is usually a function of the orientation θ : $T_r = T(\theta)$. In practice, we can simply adopt a RRPN [22] or R-CNN to learn the orientation θ of an RRoI, as well as the transformation T_r . Finally, the rotation-invariant feature $\Phi(I_R)$ can be obtained by applying an inverse transformation T_r' to Eq. 6:

$$\Phi(I_R) = T_r' \Phi(T_r I_R). \quad (7)$$

4. Rotation-equivariant Detector

This section presents details of the proposed Rotation-equivariant Detector (ReDet) to encode both rotation equivariance and rotation invariance. First, we adopt rotation-equivariant networks as the backbone to extract rotation-equivariant features. As discussed before, directly applying the RRoI Align to rotation-equivariant feature maps cannot obtain the rotation-invariant features. Therefore, we design a novel Rotation-invariant RoI Align (RiRoI Align), which produces RoI-wise rotation-invariant features from rotation-equivariant feature maps. The overall architecture of ReDet is shown in Fig. 3. For an input image, we feed it to the rotation-equivariant backbone. Then we adopt RPN to generate HRoIs, followed by an RoI Transformer (RT) [7] that transforms HRoIs to RRoIs. Finally, the RiRoI Align is adopted to extract rotation-invariant features for RoI-wise classification and bounding box regression.

4.1. Rotation-equivariant Backbone

Modern object detectors usually adopt deep CNNs as the backbone to automatically extract deep features with enriched semantic information, *e.g.*, the widely used ResNet [12] with Feature Pyramid Network (FPN) [17]. We also adopt ResNet with FPN as the baseline and implement a rotation-equivariant backbone, named Rotation-equivariant ResNet (ReResNet) with ReFPN.

Specifically, we re-implement all layers of the backbone with rotation-equivariant networks based on `e2cnn` [32], including convolution, pooling, normalization, non linearities, *etc.* Considering the computational budget, ReResNet and ReFPN are only equivariant to the discrete group $(\mathbb{R}^2, +) \rtimes C_N$, *i.e.*, all translations and N discrete rotations. As is shown in Fig. 3 (b), we can feed an image to the rotation-equivariant backbone to produce rotation-equivariant feature maps. Unlike ordinary feature maps, the rotation-equivariant feature maps f with the size (K, N, H, W) have N orientation channels: $f = \{f^{(i)} | i \in \{1, 2, \dots, N\}\}$, and feature maps of each orientation channel $f^{(i)}$ is corresponding to an element in C_N .

Compared with ordinary backbones, the rotation-equivariant backbone has the following advantages: (a) **Higher degree of weight sharing.** As we have introduced that rotation-equivariant feature maps have an additional orientation dimension. Features from different orientations usually share the same filters with different rotation transformations, *i.e.*, the rotation weight sharing. (b) **Enriched orientation information.** For an input image with a fixed orientation, the rotation-equivariant backbone can produce features from multiple orientations. This is important for oriented object detection, which requires accurate orientation information. (c) **Smaller model size.** Compared with the baseline, we have two choices when designing the backbone: similar computation or similar parameters. Typically, we keep similar computation with the baseline, *i.e.*, preserving the same output channels. Due to the rotation weight sharing, our rotation-equivariant backbone shows a huge reduction of model size, about $1/N$ of parameters.

4.2. Rotation-invariant RoI Align

As introduced in Sec. 3, for an RRoI (x, y, w, h, θ) , we can extract rotation-invariant RoI features from rotation-equivariant feature maps with RRoI warping. However, the ordinary RRoI warping can only align features in the spatial dimension, while the orientation dimension leaves misaligned. Therefore, we propose RiRoI Align to extract completely rotation-invariant features. As is shown in Fig. 3 (c), RiRoI Align includes two parts: (a) **Spatial alignment.** For an RRoI (x, y, w, h, θ) , spatial alignment warps it from feature maps f to produce rotation-invariant region features f_R in the spatial dimension, which is consistent with RRoI Align [7]. (b) **Orientation alignment.**

To ensure RRoIs with different orientations produce completely rotation-invariant features, we perform orientation alignment in the orientation dimension. Specifically, for the output region features $\hat{f}_R$, we formulate orientation alignment as:

$$\hat{f}_R = \text{Int}(SC(f_R, r), \theta), \quad r = \lfloor \theta N / 2\pi \rfloor, \quad (8)$$

where SC and Int denote the *switching channels* and *feature interpolation* operations, respectively. For the region features f_R , we first calculate an index r , and circularly switch the orientation channels to make sure $C_N^{(r)}$ is the first orientation channel. However, since the rotation equivariance is only achieved in the discrete group C_N , we also need to interpolate the feature if $\theta \notin C_N$. More precisely, we interpolate the orientation feature with its nearest l orientation channels. For example, the output feature of i -th orientation channel with $l = 2$ can be expressed as:

$$\hat{f}_R^{(i)} = (1 - \alpha)f_R^{(i)} + \alpha f_R^{(i+1)}, \quad (9)$$

where $\alpha = \theta N / 2\pi - r$ indicates the distance factor for 1D-interpolation. Note that we use the `mod` function to ensure $i \in [1, N]$ (as well as $i + 1$).

Comparison with RRoI Align+MaxPool. Different from RiRoI Align, warping RoI features with RRoI Align and then maxpooling over the orientation dimension (*i.e.*, orientation pooling) is another approach to extract rotation-invariant features. The orientation pooling operation is usually adopted in classification tasks [5, 33, 45]. For each location in the feature map, it only preserves the orientation with the strongest response, while features from other orientations are abandoned. However, we argue that the response from all orientations, no matter strong or weak, is indispensable for object recognition. In our RiRoI Align, features from all orientations are preserved and aligned with the orientation alignment operation. We will conduct experiments to show the advantage of our RiRoI Align in Sec. 5.

5. Experiments and Analysis

5.1. Datasets

DOTA [35] is the largest dataset for oriented object detection in aerial images with two released versions: **DOTA-v1.0** and **DOTA-v1.5**. **DOTA-v1.0** contains 2806 large aerial images with the size ranges from 800×800 to 4000×4000 and 188, 282 instances among 15 common categories: Plane (PL), Baseball diamond (BD), Bridge (BR), Ground track field (GTF), Small vehicle (SV), Large vehicle (LV), Ship (SH), Tennis court (TC), Basketball court (BC), Storage tank (ST), Soccer-ball field (SBF), Roundabout (RA), Harbor (HA), Swimming pool (SP), and Helicopter (HC). **DOTA-v1.5** is released for DOAI Challenge 2019³ with a new category, Container Crane (CC) and more

³<https://captain-whu.github.io/DOAI2019>

extremely small instances (less than 10 pixels). DOTA-v1.5 contains 402, 089 instances. Compared with DOTA-v1.0, DOTA-v1.5 is more challenging but stable during training.

Following the settings in previous methods [7, 10], we use both training and validation sets for training and the test set for testing. We crop the original images into 1024×1024 patches with a stride of 824. Random horizontal flipping is adopted to avoid over-fitting during training, and no other tricks are utilized. For fair comparisons with other methods, we prepare multi-scale data at three scales $\{0.5, 1.0, 1.5\}$, and random rotation for training and testing.

HRSC2016 [21] is a challenging ship detection dataset with OBB annotations, which contains 1061 aerial images with the size ranges from 300×300 to 1500×900 . It includes 436, 181 and 444 images in the training, validation and test set, respectively. We use both training and validation sets for training and the test set for testing. All images are resized to (800, 512) without changing the aspect ratio. Random horizontal flipping is applied during training.

5.2. Implementation Details

ImageNet pretrain. For the original ResNet [12], we directly use the ImageNet pretrained models from Pytorch [25]. For ReResNet, we implement it based on the `mmclassification`⁴. We train ReResNet on the ImageNet-1K with an initial learning rate of 0.1. All models are trained for 100 epochs and the learning rate is divided by 10 at $\{30, 60, 90\}$ epochs. The batch size is set to 256.

Fine-tuning on detection. We adopt ResNet [12] with FPN [17] as the backbone of the baseline method. ReResNet with ReFPN is adopted as the backbone of our proposed ReDet. For RPN, we set 15 anchors per location of each pyramid level. For R-CNN, we sample 512 RoIs with a 1:3 positive to negative ratio for training. For testing, we adopt 10000 RoIs (2000 for each pyramid level) before NMS and 2000 RoIs after NMS. We adopt the same training schedules as `mmdetection` [3]. SGD optimizer is adopted with an initial learning rate of 0.01, and the learning rate is divided by 10 at each decay step. The momentum and weight decay are 0.9 and 0.0001, respectively. We train all models in 12 epochs for DOTA and 36 epochs for HRSC2016. We use 4 V100 GPUs with a total batch size of 8 for training and a single V100 GPU for inference.

5.3. Ablation Studies

In this section, we conduct a series of ablation experiments on DOTA-v1.5 test set to evaluate the effectiveness of our proposed method. Note that we use the original ResNet+FPN and RRoI Align as the backbone and RoI warping method for the baseline method, respectively.

Rotation-equivariant backbone. We evaluate the effectiveness of rotation-equivariant backbone with ReRes-

backbone	group	cls. (%)	det. (%)	size (Mb)
R50-FPN	-	76.55	65.03	103
ReR50-ReFPN	C_4	72.81	65.43	24
ReR50-ReFPN	C_8	71.20	66.86	12
ReR50-ReFPN	C_{16}	61.60	64.36	6

Table 1. **Performance comparisons of the rotation-equivariant backbone on classification (cls.) and detection (det.).** group indicates the rotation group that the backbone is equivariant to. We report the top-1 accuracy on ILSVRC 2012 without FPN and the detection performance on DOTA-v1.5 test set in terms of mAP. The model size only includes the size of the backbone.

method	backbone	mAP (%)	size (Mb)
FR-O	R50-FPN	62.00	158
	ReR50-ReFPN	62.36	68
RetinaNet-O	R50-FPN	58.74	140
	ReR50-ReFPN	59.64	34

Table 2. **The performance of rotation-equivariant backbone on other detectors.** Faster R-CNN OBB (FR-O) and RetinaNet OBB (RetinaNet-O) are our re-implemented version for OBBs.

method	#interpolate	mAP (%)
RRoI Align	-	65.99
RRoI Align+MP	-	64.60 (-1.39)
RiRoI Align	1	66.44 (+0.45)
RiRoI Align	2	66.86 (+0.87)
RiRoI Align	4	66.32 (+0.33)

Table 3. **Comparisons of our RiRoI Align with RRoI Align.** #interpolate indicates the number of orientation channels used for interpolation (same as l in Sec. 4.2). For an RRoI with the orientation θ , we use its nearest $\{1, 2, 4\}$ orientation channels to interpolate its features. MP is short for MaxPool. ReR50+ReFPN is adopted as the backbone.

method	rot.	schd.	mAP (%)	training (h)
ReDet	$\times$	1x	62.62	8
baseline	$\checkmark$	1x	64.07	11
ReDet*	$\times$	1x	66.66	13
baseline	$\checkmark$	2x	67.34	22

Table 4. **Comparison with rotation augmentation.** We compare the performance of the baseline method with rotation (rot.) augmentation and ReDet without rotation augmentation. ReDet* preserves a similar amount of parameters with the baseline. We report the mAP with R18 (for baseline) and ReR18 (for ReDet) backbone under the cyclic group C_8 . For fair comparison, we randomly select rotation angles from $\{0, 45, 90, \dots, 315\}$.

method	DOTA-v1.0			HRSC2016		
	AP50	AP75	mAP	AP50	AP75	mAP
baseline	75.62	48.37	46.13	90.18	80.48	68.17
ReDet	76.25	50.86	47.11(+0.98)	90.46	89.46	70.41(+2.24)

Table 5. **Performance of the proposed ReDet on other datasets.** We report the performance on DOTA-v1.0 and HRSC2016 in COCO style. We use ReR50+ReFPN (*resp.* R50+FPN) as the backbone of ReDet (*resp.* baseline).

Net50+ReFPN under different settings. As shown in Tab. 1, compared to ResNet50, ReResNet50 achieves lower clas-

⁴<https://github.com/open-mmlab/mmlclassification>